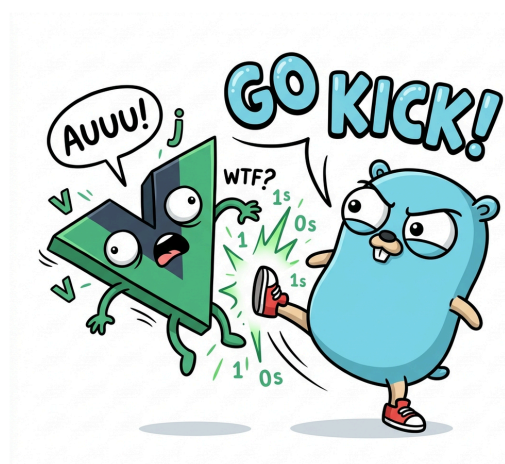


Jak dobrý, výkonný a bezpečný je gokick?

Nezávislý audit reálného kódu (nikoli dokumentace) napříč bezpečností, architekturou, výkonem, dokumentací i frontendem — se srovnáním vůči špičkovým frameworkům.



Souhrnný verdikt. Stack výrazně přesahuje úroveň běžného boilerplate — po architektonické i bezpečnostní stránce jde o nadprůměrné dílo, které v řadě disciplín převyšuje výchozí nastavení mainstreamových frameworků (Rails, Laravel, NestJS). Jediným zásadním stropem je vědomě zvolená **single-node SQLite** — optimální pro latenci a provozní jednoduchost, bez horizontálního škálování.

Toto je **aktualizované vydání**: od původního auditu (6/2026) přibyl zapínatelný multitenancy s platformní rovinou, konvergováný durable engine, velký cleanup se staticky vynucenými limity složitosti a rozšířené testy — vážené skóre se posunulo z 8,5 na 8,9.

8,9 / 10 celkové vážené skóre

PŘEDMĚT

Go DDD/CQRS + Vue 3 SPA

ROZSAH KÓDU

36 582 řádků (456 souborů)

METODA

Re-audit reálného kódu (baseline 3× audit)

DATUM

20. 7. 2026

01 Shrnutí & skóre

Původní hodnocení (6/2026) vzniklo ze tří nezávislých auditů, které procházely **skutečný kód**. Toto vydání je jeho **re-audit** vůči současnému kódu — po přírůstku multitenancy, konvergovaného durable engine, velkého cleanupu a rozšíření testů. Skóre a váhy po dimenzích níže.

Jak se počítá celkové skóre

Vážený průměr = $\Sigma(\text{skóre} \times \text{váha}) / \Sigma(\text{váha}) = 66,5 / 7,5 = 8,9$ (dříve 8,5). Všechny dimenze mají váhu $\times 1,0$; jediná výjimka je **škálovatelnost** $\times 0,5$ — vědomě úzký strop, který by s plnou vahou zkresloval celkový obraz. Prostý nevážený průměr by byl 8,6.

Pravidlo: skóre měří **kvalitu** $\times$ **šíři toho, co reálně existuje**; „cesta k 10“ (kap. 11) je pak výčet toho, co ještě chybí. Nová dobře postavená schopnost proto zvedne skóre, i když otevřené položky zůstávají.

Bezpečnost

auth, tokeny, lockout, hlavičky, audit, tenant izolace



9,5/10

$\times 1,0$

Architektura & čistota kódu

DDD, CQRS, fail-closed, vynucené limity složitosti



9,5/10

$\times 1,0$

Tooling / DX / statická vynutitelnost

lint, arch-lint, codegen, typová hranice BE $\leftrightarrow$ FE



9,5/10

$\times 1,0$

Výkon — latence, single-node

Go runtime + in-proc SQLite, stránkované listy



9,0/10

$\times 1,0$

Frontend

Vue 3, striktní TS, DataGrid, runtime-validovaný fetch



9,0/10

$\times 1,0$

Dokumentace & AI skills

29 gk-* skillů + štihlé Documan docs



10/10

$\times 1,0$

Testovací pokrytí

$\approx 1,9:1$ LOC Go, proces-E2E hotové, chybí browser-E2E/load



8,0/10

$\times 1,0$

Škálovatelnost — horizontální

vědomý strop, ne pro multi-node



4,0/10

$\times 0,5$

Největší síla

Bezpečnostní vrstva na úrovni dedikovaného security týmu: rotate refresh tokenů s **detekcí krádeže**, atomický account lockout, timing-safe login, kompletní CSP/HSTS, audit mimo transakci a **fail-closed tenant izolace** vynucená per-dotaz gate v CI. Přísnější než výchozí nastavení velkých frameworků.

Největší limit

Single-node SQLite (single-writer, ~stovky–nízké tisíce zápisů/s) a scheduler **bez leader election** (na více instancích běží cron joby opakovaně). Nejde o vadu, nýbrž o záměrné rozhodnutí — jediná binárka, žádná externí infrastruktura.

Jak číst tento dokument

Kapitoly 2–9 rozebírají jednotlivé dimenze s konkrétními důkazy z kódu. Kapitola 10 zasazuje gokick do kontextu Rails/Laravel/Spring/NestJS, kapitola 11 vymezuje **konkrétní cestu k 10/10** v jednotlivých disciplínách, kapitola 12 přináší kvantitativní data o rozsahu kódu.

02 Bezpečnost

9,5 / 10 – to nejsilnější

Bezpečnostní vrstva převyšuje výchozí nastavení Laravelu, Railsu i Expressu. Bez známých kritických zranitelností; patrné promyšlené threat modelování.

Hesla & login

- bcrypt cost 12 + **SHA-256 prehash** kvůli 72bytovému limitu bcryptu
- Proti unknown useru se počítá **dummy hash** → žádná timing user-enumeration
- Lock check až **po** Verify → časování nic neprozradí

Refresh tokeny

- crypto/rand, v DB jen **SHA-256 hash** (ne plaintext)
- **Rotace + detekce krádeže**: reuse i concurrent-race → odhlášení všech zařízení
- HS256 access token (15 min), refresh 7 dní

Account logout

- 5 pokusů / 10 min → 15 min lock
- Celé **atomicky v SQL** (`UPDATE ... RETURNING`), bez race
- Raw pool → přežije rollback business transakce

HTTP hardening

- CSRF přes **stdlib Go 1.25** (`Sec-Fetch-Site`)
- Kompletní hlavičky: **CSP, HSTS, COOP/CORP**, Permissions-Policy
- Per-IP rate limit (token bucket), maskování tajemství v logu

Multitenancy – fail-closed izolace nově

Zapínatelný row-level multitenancy (`APP_MULTITENANCY`): každý dotaz na tenant-vlastněnou tabulku musí být scopovaný, jinak **neprojde konformním testem v CI** (i neklasifikovaná tabulka padá). Resolver chybějící tenant v multitenant režimu **fail-closed** odmítne. Platformní rovina (role `superadmin`) má cross-tenant přístup; roli `superadmin` API odmítá na každé úrovni – vzniká jen mimo aplikační vrstvu (CLI/seed).

Nadstandardní opatření

Detekce **reuse rotovaného tokenu** i **concurrent-rotation race** spustí `theft_detected` a tvrdé odhlášení ze všech zařízení. 100 % parametrizované dotazy, JWT secret ≥ 32 znaků vynuceno, `alg`-confusion ošetřen. Audit log běží **mimo transakci**, takže bezpečnostní eventy (login_failed, account_locked) přežijí rollback.

Dílič poznámky (nikoli zranitelnosti): refresh token má ~119 bitů entropie (dostačující); HS256 je symetrický – vyhovující pro jednu službu, pro multi-service nasazení by byl vhodnější asymetrický RS256.

03 Architektura & kvalita kódu

9,5 / 10 — senior-level

DDD + CQRS jsou zde implementované **reálně, nikoli jako deklarace na papíře**. Vzory jsou funkční, pokryté testy a vynucené na více úrovních.

4 vrstvy vynucené `go-arch-lint`

Bounded kontexty (`user` / `token` / `run` / `tenant`) jsou izolované tak, že cross-context import **neprojde lintem** — každý kontext je vlastní arch-lint komponenta. Doménová vrstva importuje jen stdlib + uuid. Pravidlo závislostí `presentation → application → domain ← infrastructure` je vynucené staticky, ne jen domluvené.

CQRS sběrnice

- Generická, type-safe (`bus.Exec[R]`), bez reflexe
- Audit **mimo** tx, DispatchEvents **obaluje** tx → eventy až po commitu (outbox-like)
- Korektnost **dokázaná** integračními testy

Fail-closed permissions

- Command bez deklarace `Permissioned` / `SkipPermission` **spadne na sběrnici**
- Bezpečnější než spoléhání na nezapomenutý middleware
- Jeden zdroj pravdy odvozený z kódu (BE i FE)

Staticky vynucená disciplína — nejen konvence

Jediná logovací cesta (depguard allow-list + forbidigo + sloglint) fyzicky **nedovolí** `fmt.Println`, druhý logger ani raw-string klíč; korelace přes `trace_id` / `user_id`. Po auditu k tomu přibýly **vynucené limity složitosti** (funlen/gocognit/nestif, délka souboru, max 5 parametrů) a **typová hranice BE↔FE** — každý wire payload je anotovaný struct, žádné inline mapy ani `any`. Takovou disciplínu nenajdeme téměř nikde.

Vyvážená výhrada

Produkční Go mezitím vyrostl na ~9,5 k řádků o reálné schopnosti (multitenancy, durable engine, platformní rovina), takže dřívější výhrada k „vysokému poměru ceremonie k funkcionalitě“ **slábne**. Co ale platí dál: **registry doménových event/run handlerů jsou stále prázdné** — infrastruktura je připravená, ale bez produkčních odběratelů (proto zůstává v „cestě k 10“).

04 Výkon

9,0 / 10 – výborný na jednom uzlu

Na jednom uzlu je výkon výborný – Go runtime bez overheadu spravovaných běhových prostředí a SQLite přímo v procesu, bez síťového hopu.

Latence / single-node

- Go runtime: žádný JVM/PHP/Node overhead
- SQLite **in-process**, bez síťového hopu
- WAL + `_txlock=immediate` (ošetřený `BUSY_SNAPSHOT`)
- Partial indexy přesně na claim-query joby

Durable engine – nadprůměrně robustní

- Atomický `claim` (`UPDATE...RETURNING`), lease + heartbeat, owner-token fencing, exponenciální backoff
- Běží **mimo transakci** (vynuceno) – dlouhá práce nedrží SQLite write-lock; at-least-once, checkpoint + resume po pádu
- Bez externího brokeru; korektnost na úrovni River/Asynq (jen ~1 s polling)

Co ještě zbývá na jednom uzlu

Dřívější strop „`FindAll` načítá celou tabulku“ je **vyřešen** – všechny list/grid dotazy jdou přes stránkované `FindPage` (`LIMIT/OFFSET`) a `FindAll` už nemá produkčního volajícího. Zbývá **keyset stránkování** (offset u hlubokých stránek stále skenuje zahazované řádky) a cache pro hot reads. Jinak žádné N+1 ani horké dotazy bez indexu. Horizontální strop řeší následující kapitola.

05 Škálovatelnost

4,0 / 10 – vědomý strop

Horizontální škálování je jediný zásadní strop celého stacku – a je vědomě zvolený. Detailní cesta k jeho odstranění je v kapitole 11.

Kde je strop

- SQLite single-writer (~stovky–nízké tisíce zápisů/s)
- Scheduler **bez leader election** → na více instancích běží cron joby opakovaně
- Durable engine = polling po 1 s (ne LISTEN/NOTIFY)
- Rate-limit stav je in-proc → instance nejsou plně stateless

Záměrné rozhodnutí – řeší se výměnou adapteru, ne přepisem

Jediná binárka + volume, žádný Redis/Postgres/broker – ideální pro Dokploy a malá až střední nasazení. Protože perzistence sedí za doménovými `Repository` interface, není strop přepis aplikace, ale **výměna adapteru**.

Preferovaná cesta: adaptér na Postgres (+ River pro joby, leader election pro scheduler). **Alternativa se SQLite (HA + read-scale):** Turso/libSQL (nově i concurrent writes), rqlite, dqlite. Konkrétní kroky v kapitole 11.

06 Tooling / DX

9,5 / 10 – špička

Statická disciplína

- golanci-lint: depguard **allow-list**, forbidigo, sloglint + limity složitosti (funlen/gocognit/nestif)
- go-arch-lint (vrstvy i kontexty); vlastní gaty `boundary` / `errfields` / `docpaths`
- ESLint strictTypeChecked + vue-tsc + knip; **no-as** ráčna

Build & DI

- Wire = **compile-time DI**, žádná runtime magie
- Goose migrace embedované, auto-aplikace při startu
- `gk tsngen` codegen: TS typy + guardy z Go DTO (jeden zdroj pravdy)

07 Frontend

9,0 / 10

Bezpečný auth model

- Access token **jen v paměti** (nelze odcizit přes XSS)
- Refresh v HttpOnly cookie
- **Single-flight refresh** + self-healing session

Čistota, typy & DataGrid

- Vue 3 `script setup`, striktní TS; **runtime-validovaný fetch** (2xx tělo ověřené proti kontraktu)
- Server-side **DataGrid**: stránkování/filtry/řazení, výběr + bulk akce, URL sync
- Router guardy zrcadlí backend permissions; žádná FE validace

Stav je řešen záměrně lehce – manuální reactive refs plně pokrývají současný rozsah aplikace. Plnohodnotný state management (Pinia) je volbou pro budoucí růst, nikoli chybějícím prvkem.

08 Dokumentace & AI skills

10 / 10 – referenční

Dokumentace je vědomě dvouvrstvá: **štíhlé lidské docs** (Documan) + **29 gk-* skillů** jako strojově i lidsky čitelný how-to. Jde o AI-native přístup, který je v současné praxi vzácný – proto byl vyčleněn jako samostatná hodnocená dimenze.

29 gk-* skillů

- Jeden skill = jeden koncept (auth, repos, runs, permissions, multitenancy, observability, deploy...)
- Každý: **proč/kdy** · **vysvětlení** · **jak funguje** · **recept** · **časté chyby**
- Psané pro Claude Code i juniora / non-tech čtenáře

Jednotný vstup `/gk`

- Rozcestník `/gk` nebo přímo `/gk-<téma>`
- AI si skill vybere sama podle `description`
- Šablona `_TEMPLATE.md` vynucuje strukturu (6 sekcí, CZ tělo / EN nadpisy)

Štíhlé lidské docs (Documan)

- architecture · installation · configuration · sekce HTTP/CQRS/Background · roadmap
- Verbose per-layer docs **vědomě odstraněny** ve prospěch skillů
- Gate `docpaths` : každá citovaná cesta i `/gk-*` odkaz musí existovat, jinak padá CI

`CLAUDE.md` jako kontrakt

- Invarianty, checklist featur, pravidla vrstev na jednom místě
- Drží AI agenta i člověka ve stejných mantinelech
- Mapováno 1:1 na lint/arch pravidla – dokumentace = vynucení

Hodnocení dimenze

Dokumentace primárně určená pro **AI agenta**, již zároveň bez problémů přečte junior, je v současné praxi vzácná – většina projektů spoléhá na zastaralou wiki, nebo nemá nic. Zde je navíc provázána s kódem přes lint a arch pravidla. ADRs jsou součástí šablony, nikoli tohoto projektu, a proto se do hodnocení nepromítají – dimenze je naplněna v plném rozsahu.

09 Testovací pokrytí

8,0 / 10 – proces-E2E hotové, chybí browser/load

Pokrytí je svým rozsahem nadprůměrné a integračně těžké; nejvíc chybí browserové E2E a zátěžové testy.

Co je solidní

- Poměr **testy:produkce u Go** $\approx 1,9:1$ (18 093 vs 9 487 řádků)
- **testfx harness** nad reálnou SQLite – ne mocky
- **Proces-lifecycle E2E**: crash-recovery, at-least-once, SIGTERM drain, terminální selhání
- Arch-konformní testy (`zz_audit` / `zz_gap` , tenant gate) + běh s `-race`

Co chybí do 10

- **Browser-E2E** (Playwright) – proces-E2E hotové, browserové flow ještě ne
- **Zátěžové / výkonové** testy (k6, vegeta)
- **Coverage gate** v CI + mutation/contract testy

Hodnocení dimenze

CI (`validate.yml`) na každý PR spouští lint + test + build + **paralelní proces-E2E job**, takže regrese i pády procesu se chytají brzy. Skóre drží zpět **chybějící vrstvy** – browserová a zátěžová – a absence coverage gate. Doplnění je přímočaré (kapitola 11).

10 Srovnání s TOP řešeními

ŘEŠENÍ	GOKICK VYHRÁVÁ NA	GOKICK PROHRÁVÁ NA
Rails / Laravel / Django	type-safety, startup, paměť, single-binary deploy, přísnější bezpečnostní defaulty, row-level multitenancy na přepínač	ekosystém, šíře hotových funkcí, scale-out příběh, komunita
Spring Boot / NestJS	žádný runtime overhead JVM/Node, compile-time DI bez magie za běhu	obří ekosystém integrací, hotové enterprise moduly
Typický Go web projekt	v horních jednotkách procent – disciplína, jakou má málokdo	– (medián je plochý <code>handler + sqlc</code>)

Celkové zarámování

Velké frameworky stojí na principu „baterie + magie + komunita“; gokick na principu „nula magie, plná kontrola, ručně udržované“. Go runtime přináší výbornou latenci a nízkou paměťovou stopu oproti JVM/PHP/Python/Node; pro read-heavy nebo středně zápisové workloady na jednom uzlu je rychlejší a úspornější než kterýkoli z velkých frameworků.

Jako foundation/boilerplate jde prakticky o best-in-class řešení pro malá až střední nasazení s důrazem na bezpečnost, čistotu a jednoduchý provoz. Pro multi-node nebo vysoký zápisový throughput určen není – a ani si to neklade za cíl.

Silné stránky

Bezpečnostní vrstva nad rámec defaultů frameworků.

Reálné, testované DDD/CQRS s lint-enforcementem.

Poměr testy:produkce u Go $\approx 1,9:1$.

Zapínatelný row-level multitenancy + platformní rovina.

Single-binary deploy, embedované migrace i SPA.

11 Cesta k 10/10

Co konkrétně chybí do plného skóre v jednotlivých disciplínách. **Jedna disciplína nese ~90 % práce** – škálovatelnost; zbytek představují cílené dílčí úpravy.

Škálovatelnost 4 → 10 – sem patří většina práce

A) Zůstat u SQLite – HA + read-scale

- **Turso / libSQL** – embedded replicas, nově i concurrent writes (MVCC)
- **rqlite** – Raft, nejzralejší clustering; **dqlite** (Canonical)
- LiteFS je funkční, ale pre-1.0 a deprioritizovaný → pro nové projekty vhodnější Turso

B) Skutečný write-scale – Postgres

- **Výměna adapteru, ne rewrite** – `infrastructure/postgres/*` + `wire.Bind`
- Durable frontu → **River** (Postgres-native, battle-tested)
- Scheduler → **leader election** (advisory locks); rate-limit stav → Redis

Bezpečnost 9,5 → 10

- 2FA (TOTP) + WebAuthn
- refresh 256-bit + EdDSA JWT pro multi-service
- CSP bez `unsafe-inline` (nonce); gosec/CodeQL/govulncheck v CI

Architektura 9,5 → 10

- zapojit prázdné event/run registry reálnými handlers
- netriviální bounded context mimo auth jako vzor

Tooling / DX 9,5 → 10

- coverage gate + mutation testing v CI
- Dependabot/Renovate + govulncheck
- SBOM + podepsané release (cosign)

Výkon 9,0 → 10

- keyset stránkování list dotazů
- cache vrstva pro hot reads
- benchmarky + pprof profily

Frontend 9,0 → 10

- E2E (Playwright)
- a11y audit (axe)
- i18n

Testovací pokrytí 8,0 → 10

- browser-E2E + load (k6/vegeta)
- coverage gate v CI
- contract + mutation testy

Výsledek

Mimo škálovatelnost jsou všechny dimenze otázkou **týdnů cílené práce**, nikoli přestavby. Architektura tento růst již předpokládá.

12 Statistiky kódu

Měřeno nástrojem `cloc`. Sloupec „kód“ je čistý kód bez komentářů i prázdných řádků.

15 863 řádků

PRODUKČNÍ KÓD

Go 9 487 · frontend 5 991 · SQL 80 · generovaný DI 304

20 719 řádků

TESTOVACÍ KÓD

Go 18 093 · TypeScript 2 467 · shell 159 — poměr Go $\approx$ 1,9:1

SLOŽENÍ PRODUKČNÍHO KÓDU

Go 9 487

Vue 3 917

TS 2 074 · SQL 80 · DI 304

Go (backend) Vue komponenty TypeScript / SQL / generovaný DI

KATEGORIE	SOUBORY	KÓD	KOMENTÁŘE	PRÁZDNÉ
Go — produkce (bez testů a wire_gen)	141	9 487	3 063	1 596
Go — testy (*_test.go)	132	18 093	2 956	2 606
Go — generovaný (wire_gen.go)	1	304	59	33
Vue (komponenty + views)	60	3 917	190	464
TypeScript — produkce	82	2 074	570	420
TypeScript — testy	32	2 467	335	803
SQL (migrace)	2	80	76	12
Shell (E2E harness)	5	159	53	29
CSS	1	1	0	0
Celkem	456	36 582	7 302	5 963

Co z čísel plyne

Poměr **testy:produkce u Go $\approx$ 1,9:1** zůstává silným signálem kvality — mírný pokles z 2,15:1 vznikl tím, že **produkční kód mezitím vyrostl rychleji** (multitenancy, durable engine, platformní rovina) než testy, ne poklesem testovací kázně. Hustota komentářů v produkčním Go $\approx$ 24 % — komentáře jsou „load-bearing“ (vysvětlují proč), ne výplň. Nad rámec této tabulky žije ještě **~2 750 řádků vlastního Go toolingu** ([tools/gk](#) : codegen + lint gaty).

13 Závěr

Jako **foundation/boilerplate je gokick výjimečný** — po bezpečnostní i architektonické stránce nad úrovní výchozího nastavení velkých frameworků a výrazně nad mediánem Go projektů. Strop je jediný a vědomý: single-node SQLite, bez horizontálního škálování.

Pro malá až střední nasazení s důrazem na **bezpečnost, čistotu a jednoduchý provoz** jde prakticky o best-in-class řešení. Pro multi-node nebo vysoký zápisový throughput stavěný není — a ani si to neklade za cíl.

K tomu **AI-native dokumentace** (29 gk-* skillů) posouvá laťku tam, kde ji většina projektů nemá vůbec. Mimo škálovatelnost je **cesta k 10/10 otázkou týdnů cílené práce** (kapitola 11), nikoli přestavby — architektura tento růst již předpokládá.

8,9 / 10

CELKOVÉ VÁŽENÉ SKÓRE

0

ZNÁMÝCH KRITICKÝCH ZRANITELNOSTÍ